

[illegible]

INGA STRØMKE

**MASKINER
DER TÆNKER**

Algitmernes hemmeligheder
og vejen til kunstig intelligens

POLITIKENS FORLAG

INGA STRØMKE

**MASKINER
DER TÆNKER**

Algitmernes hemmeligheder
og vejen til kunstig intelligens

POLITIKENS FORLAG



INGA STRØMKE

**MASKINER
DER TÆNKER**

Algoritmernes hemmeligheder
og vejen til kunstig intelligens

POLITIKENS FORLAG

Del 1

BEGYNDELSEN PÅ DET HELE

Kapitel 1

En smule nøje udvalgt historie

Verdens mest vellykkede svindel

Ærkehertuginde Maria Theresia af Østrig må have været noget af en kvinde. For at imponere hende byggede opfinderen Wolfgang von Kempelen det, der senere blev kendt som verdens mest vellykkede svindelnummer: Den mekaniske tyrker. Dette vidunder af en maskine skabte allerede i 70'erne de første overskrifter om maskiner, der var bedre end mennesker til at spille skak. Altså i 1770'erne. Maskinen bestod af en mekanisk mand iklædt kappe og turban, som sad på en kasse, der indeholdt et virvar af tandhjul og tilsyneladende kompliceret maskineri. "Tyrkeren" kunne spille skak mod mennesker, og spillet begyndte ved, at den – eller han? – bevægede hovedet fra side til side, som om han betragtede skakbrættet og planlagde sit første træk. Så skød hans arm pludselig frem, og de mekaniske fingre tog en brik op og placerede den på et lovligt felt. Det var ikke i sig selv nok til at imponere folk i 1700-tallet, for de havde set mekaniske dyr med langt mere overbevisende detaljer end denne grove efterligning af et menneske. Det banebrydende var selve spillet: Tyrkeren var god til skak. Virkelig god. Han slog selveste Napoleon Bonaparte, som gik i graven i den tro, at en maskine havde besejret ham i skak. Tyrkeren spillede kreativt og tilpassede sig modstanderens stil, parti efter parti. Og hvis nogen forsøgte at snyde, sådan som Napoleon gjorde, da han

udfordrede Tyrkeren i 1809, flyttede Tyrkeren brikken tilbage til der, hvor den kom fra. Hvis modstanderen blev ved med at forsøge at snyde, mistede Tyrkeren til sidst tålmodigheden og fejede armen hen over brættet, så brikkerne fløj til alle sider.

Det hele var naturligvis bare et bedrag, men i form af et imponerende stykke ingeniørkunst: Den sindrige kasse med alle tandhjulene kunne åbnes og inspiceres, og før hvert parti fik publikum lov til at kigge ind i den fra alle sider. Derfor var tilskuerne altid overbevist om, at der ikke befandt sig et menneske i kassen.¹ Spekulationerne var alligevel mange og inkluderede, at den egentlige hjerne i Tyrkeren var en russisk soldat med talent for skak, der havde mistet begge ben i kamp. Da det hele til sidst blev afsløret i *The Chess Monthly* i slutningen af 1850'erne, blev det sagt, at "ingen hemmelighed er nogensinde blevet bevaret så godt som den om Tyrkeren". Fidusen var, at det menneske, der selvfølgelig befandt sig inden i kassen, sad på en stol, som kunne bevæge sig og dermed undgå at blive set under inspektion.

At en skakspillende maskine fik så meget opmærksomhed, er der nok flere grunde til, men blandt dem var idéen om, at en maskine kunne være klogere end mennesker; træffe *bedre beslutninger* end mennesker. Og selvom verden måtte vente længe endnu på en maskine, som kunne spille skak på niveau med mennesker, var maskiner allerede i 1800-tallet godt i gang med at forandre både folks liv og hele samfundsøkonomien.

Dampmaskinen er blevet et ikon for 1700-tallets opfindelser af maskiner, der gjorde energi tilgængelig på en måde og i et omfang, som hverken dyr, mennesker eller datidens teknologi kunne hamle op med. Overgangen fra menneskeligt håndværk til maskinel produktion begyndte i England, spredte sig hurtigt gennem Europa og til USA og blev et vendepunkt i moderne industri. I stedet for at mennesker producerede varer, blev der bygget maskiner, som producerede dem mere effektivt, og den jordbrugsbaserede økonomi blev i hastigt tempo afløst af en industriel

og maskinbaseret økonomi. I den vestlige verden, hvor langt de fleste var bønder og levede af jorden, medførte det en total omvæltning i levevis. Og Den mekaniske tyrker blev berømt midt i denne tid. Så det er ikke underligt, at tanken om en maskine, som ikke bare kunne arbejde, men også *tænke* mere effektivt end mennesker, vakte både fascination og bekymring. For selvom Tyrkeren ikke direkte var intelligent, passede den ind i forestillingen om en mulig fremtid, hvor maskiner kunne tænke selv og måske ende med at erstatte mennesker.

Desværre, eller måske heldigvis, viste det sig dog, at Tyrkeren havde mere tilfælles med dampmaskinen end med mennesker. I og med at den ikke selv tog sine skakbeslutninger, var den ikke andet end et mekanisk værktøj, der kunne flytte brikker. Både Tyrkeren og dampmaskiner har brug for mennesker til at betjene sig og er ikke selv i stand til at træffe beslutninger.

Selvom der skulle gå lang tid, før maskiner, der træffer beslutninger på egen hånd, blev en del af virkeligheden, kan jeg godt lide at tro, at 1800-tallets mennesker alligevel lod sig fascinere af tanken. Måske læste de om "Maskinen" ("The Engine") i *Gullivers rejser* af Jonathan Swift, og måske havde de stærke meninger om, hvorvidt det lod sig gøre at bygge en *tænkende maskine*.² Og måske fulgte de allerivrigste af dem med i de nyeste teknologiske udviklinger og registrerede det, da verdens første *algoritme* allerede i begyndelsen af 1800-tallet så dagens lys.

Algoritmer

Ja, algoritmer fandtes længe før kunstig intelligens. Selvom ordet i dag ofte bruges i medierne til at beskrive software, som analyserer vores data eller får os til at gøre bestemte ting, er algoritmer meget mere end bare dét. Kort fortalt er en algoritme en samling instruktioner, som kan udføres i en bestemt rækkefølge for at opnå et bestemt mål, og det klassiske eksempel er en madopskrift. Hvis opskriften ikke indeholder alle de nødvendige

instruktioner, kommer der ikke en velsmagende ret ud af det. Hvis instruktionerne kommer i forkert rækkefølge, bliver opskriften også umulig at følge. Vi mennesker er ganske gode til at lave algoritmer til hinanden, og hver gang du skal beskrive, hvordan noget skal gøres, laver du en algoritme. Hvis jeg spørger, om jeg må låne noget mel af dig, kan det være, at du svarer: ”Ja, det må du gerne, men jeg er ikke hjemme, så du må selv låse dig ind. Brug nøglen, som ligger under måtten, gå ud i køkkenet, og kig i den nederste skuffe ved siden af køleskabet.” På den måde vil du gøre mig i stand til at låne mel af dig, og jeg vil være dig taknemmelig for at have lavet den algoritme, som gjorde det muligt.

Algoritmer er det, der får computere til at fungere, og ingen computer kan fungere uden instruktioner fra algoritmer. At lave gode algoritmer til computere er langtfra enkelt, fordi de skal have hvert eneste skridt skåret ud i pap, og fordi den bedste måde at løse et problem på ikke er indlysende. Hos de it-virksomheder, der er eftertragtede arbejdspladser, handler spørgsmålene til jobsamtalen ofte om at beskrive eller lave gode algoritmer på stedet.

At lave algoritmer er måske noget af det sjoveste, der findes, ikke fordi det er morsomt at forklare ting for computere, men fordi det er morsomt at tænke sig frem til den bedste måde at løse problemer på. Det samme syntes grevinde Ada Lovelace, som hurtigt blev led og ked af de typiske grevindeaktiviteter og i stedet viede sit liv til datalogien. I 1843 skrev hun verdens første algoritme til en ”computer” og regnes derfor som verdens første programmør. Sammen med opfinderen Charles Babbage arbejdede hun i mange år på at designe verdens første mekaniske computer. Fordi de løb tør for penge, og fordi Babbage blev uvenner med sin cheffingeniør, blev den aldrig bygget.³ Men selvom selve computeren ikke blev til noget i denne omgang, var algoritmen alligevel opfundet, og idéen om, at en maskine kan udføre en hvilken som helst proces, var født.

Computere

Tanken om computere fandtes, længe før de første computere blev bygget, og de tidligste af dem ligner på ingen måde de computere, vi bruger i dag. Den største forskel er nok, at de første computere var *analoge*: De blev bygget til at udføre én bestemt opgave. Hvis de skulle løse en anden opgave, var man nødt til at bygge dem om. Hvis en af den slags computere skulle løse et matematisk problem, blev den derfor til en fysisk manifestation af problemet – simpelthen en *analogi*. Det lyder måske abstrakt og svært at forstå, men jeg tør vædde på, at du så sent som i dag har anvendt en analog computer. Børn lærer ofte at lægge sammen og trække fra ved hjælp af farverige kuglerammer, hvor de kan flytte perler rundt på tynde stænger. Siden antikken har vi mennesker brugt disse såkaldte *abacusser* til at regne med, og mange vil måske mene, at det er overdrevet at kalde dem *maskiner*, men de illustrerer alligevel, hvordan analoge computere fungerer: De henter analoge data ind, altså fysiske størrelser direkte fra den virkelige verden. De behøver ikke blive konverteret til talværdier eller tilnærmelser, men går lige ind i maskinen. Den analoge maskine foretager sine beregninger direkte ved at måle de kontinuerlige ændringer, som sker i fysiske størrelser, og præsenterer resultatet af beregningen som en aflæsning af en måling. En miniversion af dette er et kviksølvtermometer, hvor vi ikke aflæser et tal fra et display, men i stedet foretager målingen selv ved at se, hvor højt kviksølvet er steget. Endnu mere avancerede analoge computere fra hverdagen er mekaniske armbåndsure og speedometre i biler. Hvis der er én ting, du skal huske her (fordi det bliver vigtigt senere), så er det, at programmet, på engelsk software, og selve maskinen, på engelsk hardware, er *ét og det samme*, når det gælder analoge computere.

Fordelen ved analoge computere er, at de henter deres data ind direkte, og at det derfor ikke er nødvendigt at lave målingerne om til tal med alle de fejlkilder og komplikationer, det kan medføre. Men den store svaghed ved

analoge computere er, at de kun kan løse præcis det problem, de er blevet bygget til.

Gödel, Turing og von Neumann

Nu skal vi møde tre matematikere, som var uundværlige for udviklingen af moderne datalogi, og hvis bidrag til den stadig giver forskere og studerende noget at tænke over den dag i dag: Kurt Gödel, Alan Turing og John von Neumann. I 1931 præsenterede Gödel et bevis, som rystede verden så meget, at det stadig kan mærkes i dag. Han viste, at ingen matematiske systemer kan være både komplette og konsistente, som en matematiker ville udtrykke det.⁴ Det betyder, at man ikke kan lave et logisk system, hvor alt, hvad der er sandt, kan bevises og samtidig være helt uden selvmodsigelser. Tænk bare på udsagnet ”denne sætning er en løgn”. Faktisk er det besynderligt, at man overhovedet kan lave den slags udsagn, hvis man tænker over det. Og netop dét gjorde Gödel også: Han tænkte på systemer, som laver udsagn om sig selv, og endte med at vise, at de giver problemer i matematikkens verden.

Alan Turing blev dybt fascineret af Gödels bevis og funderede over dets betydning for computere. I 1937 lykkedes det Turing at bevise to ting, som begge regnes for at være blandt de mest betydningsfulde resultater inden for datalogien. Den ene var, at det er umuligt at lave et program, som kan finde ud af, om en algoritme kan fuldføres, altså om en beregning bliver ført til ende. Dette problem kaldes *the halting problem*, og selvom det måske virker som et lidt pudsigt problem at beskæftige sig med, støder vi på det overalt. Hvis man for eksempel har et antivirusprogram og har lyst til at vide, om det nogensinde kommer til at udføre en ondsindet handling mod computeren, er man stødt på *the halting problem* og må give fortabt (hilsen Turing, anno 1937).

Beviset på, at det ikke er muligt at bevise, at en algoritme bliver ført til ende, er for resten vidunderligt finurligt og helt i Gödels ånd. Lad os

antage, at der rent faktisk findes et program, som kan finde ud af, om en algoritme før eller siden bliver færdig. Vi kan kalde dette program ”VIL_STANDSE”. Derefter kan vi skrive følgende lille program:

INGAS_ALGORITME

Hvis INGAS_ALGORITME bliver færdig:

Udfør denne instruktion uendeligt

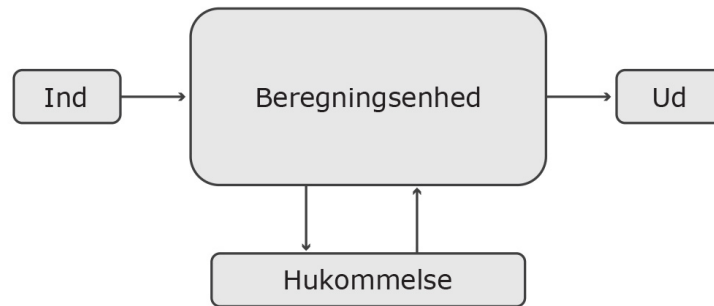
Hvad kan programmet VIL_STANDSE nu fortælle os om INGAS_ALGORITME? Hvis INGAS_ALGORITME bliver færdig, kommer INGAS_ALGORITME til at vente uendeligt – altså aldrig blive færdig. Vi kan se, at vi er kommet frem til en selvmodsigelse (og muligvis har vi fået en smule hovedpine) – og har dermed også bevist, at programmet VIL_STANDSE ikke kan eksistere.

Som det andet beviste Turing, at der kan findes maskiner, som er i stand til at udføre en hvilken som helst matematisk beregning, så længe den er formuleret som et sæt instruktioner – altså som en algoritme. Denne tænkte maskine har fået navnet *Turingmaskinen*. Med idéen om Turingmaskinen opstod tanken om et skel mellem på den ene side den maskine, som udfører beregninger, altså et stykke hardware, og på den anden side selve beregningerne, også kaldet software. Denne tanke er velkendt for os moderne mennesker, som er vant til at installere nye programmer på den gamle computer (og nye apps i ét væk på den samme telefon), men skellet mellem hardware og software var alt andet end indlysende. De eneste beregningsmaskiner, der fandtes før Turing, var den analoge computer og den menneskelige hjerne. For begge disse maskiner er hardware og software det samme. Selvom det ikke er indlysende ved første (eller andet) øjekast, udgør Turings to beviser en stor del af det fundament, moderne computere er bygget på.

Hvor genial Turingmaskinen end måtte være, er den stadig kun et abstrakt koncept. Turing beviste, at en sådan maskine i teorien *kan* findes, men som bekendt er der ofte langt fra idé til produkt. For at bygge en

maskine, der kan udføre en hvilken som helst beregning, er der behov for konkretisering. Her kommer den tredje af vores helte ind i billedet, nemlig John von Neumann. Han designede den opbygning, vi stadig bruger til at konstruere moderne computere efter, så næste gang du tænder for din PC, kan du sende John en venlig tanke og takke ham for *von Neumann-arkitekturen*. I denne opbygning, eller arkitektur, har computere én central komponent, som udfører beregninger, og en anden komponent, som står for at gemme data. Når beregninger skal udføres, skal de nødvendige ingredienser derfor hentes fra hukommelsesenheden, og hvis resultatet af en beregning skal gemmes, skal det flyttes fra beregningsenheden og ind i hukommelsen. Din computer fungerer på samme måde: Det allermeste ligger trygt og godt i hukommelsen, og kun det, der skal bruges til en beregning, bliver flyttet til beregningsenheden. Eller for at være helt præcis må jeg hellere tilføje, at der i dagens computere ligger en hel del ”grums” mellem beregningsenheden og hukommelsen, som hjælper til med at gøre både beregning og lagring af data hurtigere og enklere. Men hvis vi kniber øjnene lidt sammen, ser moderne computere stadig sådan ud, som von Neumann i sin tid tegnede dem.

Den sidste ingrediens i von Neumann-arkitekturen er to enheder, som sørger for, at mennesker kan kommunikere med maskinen. Én enhed modtager kommandoer fra et menneske og videregiver dem til beregningsenheden, mens den anden enhed henter information ud af beregningsenheden, så mennesket (eller en anden maskine, for den sags skyld) kan gøre brug af den. Figuren herunder er en simpel version af von Neumann-arkitekturen og derfor også en slags tegning af din laptop.



Von Neumann-arkitekturen var en grundlæggende ny måde at designe computere på, og den revolutionerede den moderne computervidenskab. Den beskriver en maskine, som kan modtage et hvilket som helst sæt instruktioner, så længe den forstår dem, og kan foretage beregninger på baggrund af dem. Den første von Neumann-maskine stod færdig i 1952. Samme år opfandt matematikeren Grace Hopper det første program, som gør det muligt for computere at oversætte ord, helt konkret engelske ord, til maskinsprog. Før den tid måtte man programmere computere direkte med nuller og ettaller, 0 og 1. At skrive nuller og ettaller er ikke nogen særligt morsom opgave for os mennesker, som er vant til at formulere os ved hjælp af ord. Det, Grace Hopper skabte, var verdens første såkaldte *kompilator*.

Allerede på denne tid, før computeren knap nok var opfundet endnu, grublede forskerne over, hvorvidt *præcise instruktioner til computere er nøglen til at gøre dem intelligente*. Særligt ét svar, som John von Neumann gav under en forelæsning, illustrerer hans holdning godt: En af tilhørerne havde hævdet det synspunkt, at det er umuligt for en maskine, i hvert fald en *menneskeskabt maskine*, at tænke, og von Neumann svarede: "Du insisterer på, at der findes noget, en maskine ikke kan gøre. Hvis du fortæller mig, hvad det nøjagtigt er, en maskine ikke kan gøre, kan jeg altid lave en maskine, som gør netop dét!"⁵ Det er morsomt at tænke på, at vi stadig konstruerer vores computere efter von Neumann-arkitekturen, men samtidig stadig ikke ved, om de kan rumme egentlig intelligens.

Med udviklingen af computere i midten af 1900-tallet tog kunstig intelligens under alle omstændigheder springet fra svindel- og drømmeland til den moderne videnskab via idéverdenen, og arbejdet med at lave computere, der kan tænke som mennesker, var gået i gang.

Et fagområde bliver født

Der findes ikke mange videnskabelige områder med en så præcist defineret oprindelse som kunstig intelligens. Det hele skete på en lille workshop ved Dartmouth College på den amerikanske østkyst. Den var arrangeret af den unge forsker John McCarthy, som senere blev en af de absolut mest indflydelsesrige forskere inden for kunstig intelligens. Før han begyndte på Dartmouth, studerede McCarthy matematik ved Princeton, hvor han stiftede bekendtskab med en anden kommende stjerne inden for kunstig intelligens, nemlig Marvin Minsky. De to fandt hurtigt melodien og forstod, at de havde en fælles lidenskab for spørgsmålet om computeres mulige intelligens. Da McCarthy så i 1955 blev indskrevet ved Dartmouth, overtalte han Minsky og to andre medstuderende til at hjælpe sig med at arrangere en lille workshop. Han så helt bogstaveligt for sig to måneder hen over sommeren 1956, hvor ti mennesker sammen skulle finde ud af alt det her med ”intelligente computere”. I den forbindelse fandt McCarthy også på begrebet *artificial intelligence (AI)* – kunstig intelligens – hovedsageligt for at adskille området fra det beslægtede felt kybernetik.

Forskellen mellem kunstig intelligens og kybernetik er subtil, men fundamental. Begge områder handler om at studere intelligente systemer, men hvor kunstig intelligens har som formål at skabe maskiner, der efterligner menneskelig adfærd og intelligens, handler kybernetik om at forstå, hvordan intelligente systemer – det være sig biologiske eller mekaniske – processerer information. Kort fortalt er kybernetik studiet af kommunikation og kontrol, mens det nye område *artificial intelligence* var – og er! – bestræbelsen på at skabe intelligens i et kunstigt system. Efter at

begrebet var etableret, sagde McCarthy for øvrigt, at ingen egentlig brød sig om betegnelsen *artificial intelligence*, men at der nu engang var behov for at kalde det noget.

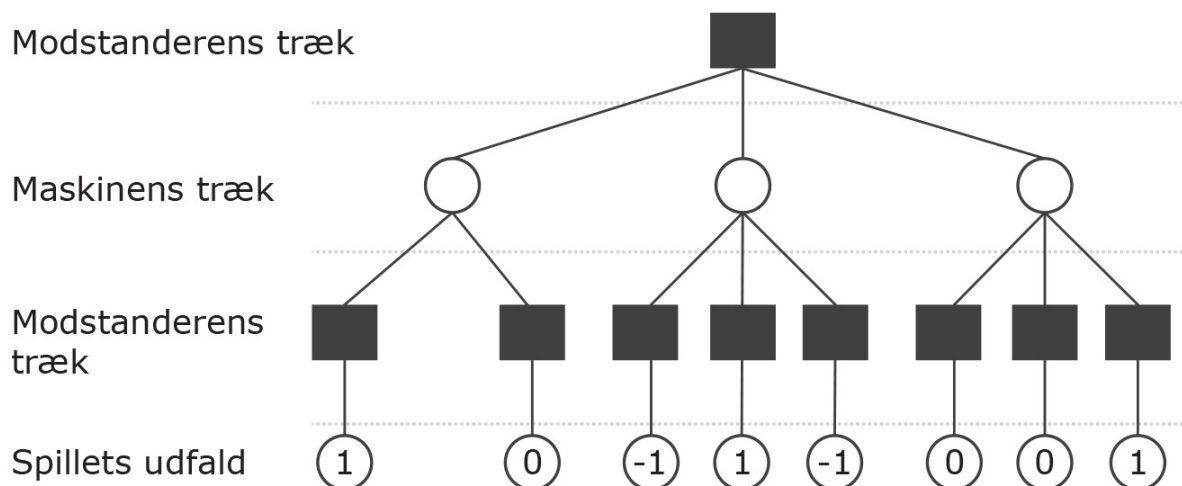
Sammen søgte de fire studerende i sommeren 1956 om forskningsmidler til at finansiere deres workshop, og i ansøgningen angav de blandt andet logisk tænkning, maskinlæring, neurale netværk, kreativitet og sprogforståelse som emner, de ville udforske. De skrev, at "ethvert aspekt ved intelligens eller læring kan i princippet udtrykkes så præcist, at en maskine kan simulere det" – og dermed var området grundlagt. Og selvom det alt sammen gik meget hurtigt, udgør det fundament, som McCarthy og hans medstuderende skabte den sommer, stadig grundlaget for selv den moderne forskning i kunstig intelligens. Den dag i dag er det stadig de samme begreber – maskinlæring, neurale netværk, sprogforståelse og så videre – vi forsker i, og den grundlæggende tanke om, at læring og intelligens kan simuleres af computere, er stadig fremherskende inden for området.

At et helt videnskabeligt område kan grundlægges og defineres for mange årtier fremover af en flok glade studerende i løbet af en sommer, må enten betyde, at der var tale om usædvanligt begavede studerende, eller at området er betydeligt vanskeligere, end de havde forventet. Sandheden rummer nok lidt af begge dele. Særligt fire af de studerende – John McCarthy, Marvin Minsky, Herbert Simon og Allen Newell – blev pionerer inden for området, og man kan roligt anse dem som "de fire store" inden for kunstig intelligens. Samtidig var der dog tale om den første af mange gange, hvor mennesker har undervurderet, hvor kompliceret vejen til kunstig intelligens skulle blive og stadig er. Gang på gang har vi troet, at vi befinder os lige på kanten til det store gennembrud, som vil få maskiner til at tænke selvstændigt, og gang på gang har vi måttet sande, at det ikke var tilfældet.

Vores kære studerende afholdt deres sommermøde og lagde planer med en enorm optimisme. Men 20 år senere, altså i 1970'erne, var vi stadig milevidt fra at løse intelligensens gåde. Der fandtes for eksempel stadig ingen computere, som kunne slå gode skakspillere. Der fandtes godt nok programmer, som kunne holde sig til reglerne og være udfordrende for amatører, men det var det hele. Faktisk skulle man helt op i midten af 1990'erne, før skakmestrene fik egentlig modstand fra maskinerne.

Skakmat

Det er nemt at få en computer til at forstå reglerne i skak. Men at lære den at vælge mellem alle de mulige træk er uhyre tungt og beregningskrævende. At mestre skak handler om at vide, hvad der kommer til at ske i fremtiden, og at vælge træk baseret på det. For at vælge sit næste træk i et parti må en skakcomputer bygge et stort søgetræ, som indeholder alle de mulige fremtidige træk fra begge spillere – den selv og modspilleren. Nedenfor er et eksempel på en lillebitte del af sådan et træ, hvor de hvide cirkler repræsenterer maskinens træk og de sorte firkanter modstanderens:



I bunden af træet finder vi udfaldene på alle de mulige partier; 1, 0 og -1 for sejr, remis og nederlag. Ved at bruge sådan et søgetræ kan en skakcomputer ved hjælp af en algoritme, som hedder *Minimax*, finde ud af, hvilket træk der er det bedste.⁶ Denne algoritme blev skabt af John von Neumann i 1928 – ja, længe før kunstig intelligens overhovedet var blevet et videnskabeligt fagområde – og tilpasset til brug i skak af Claude Shannon (som også deltog i workshoppen på Dartmouth i 1956) i 1950. Algoritmen handler om at arbejde sig fremad fra det seneste træk og vælge den bedst mulige position som svar på hver af modstanderens mulige positioner, altså maksimum. På næste niveau antager den, at modstanderen vil vælge den dårligst mulige position for en selv, altså minimum. Denne udveksling mellem maksimum og minimum har givet navn til algoritmen: Minimax.

Helt til sidst vælges den bedste af de tre højest rangerede positioner som det bedste træk. Derefter er det modstanderens tur til at foretage et træk, og ... skakcomputeren må så danne et helt nyt træ baseret på den nye position på brættet og gentage hele minimax-affæren igen. Og sådan fortsætter det træk for træk, lige til partiet er slut. Man kan roligt kalde Minimax for et kvikt påfund, men hvis man kigger på et skakbræt, får man hurtigt øje på et enormt problem: Det er umuligt at forestille sig alle de mulige spil, der kan udvikle sig på brættet, eller søge gennem hele træet, sådan som en skakcomputer er nødt til. Hvis der i gennemsnit er 35 mulige træk fra en given position, og et parti skak omfatter omkring 80 træk i alt, får man et søgetræ med 35^{80} mulige positioner. Det er et helt enormt tal. Vi kan skrive det som 10^{123} , altså et 1-tal med 123 nuller efter. Så mange positioner måtte Minimax gennemgå for at sige noget om værdien af en given stilling.⁷ Til sammenligning består hele universet, med alt interstellart støv og alle de galakser, vi kender til, af omkring 10^{75} atomer. Det betyder to ting. Den ene er, at skak som spil nok aldrig vil blive fuldstændigt løst. Vi kan ikke vide, om hvid vinder, hvis hun eller han spiller perfekt, eller om det perfekte spil skak ender i remis. Set på den måde er skak en af universets

gåder – og det er jo ganske smukt at tænke på. Den anden er, at skakcomputeren umuligt kan gennemsøge hele træet, og derfor må vi tage et par tricks i brug. Tre styk, for at være helt præcis.

Det første trick – som virkelig skiller gode skakcomputere fra dårlige – er at lave en evalueringsfunktion, som siger noget om, hvor god en position er, *uden at være nødsaget til at søge i træet*. En primitiv skakcomputer kan for eksempel sammenligne antallet af de to spilleres brikker. En lidt bedre skakcomputer tager højde for, at brikkerne er forskellige, at en dronning er mere værd end en bonde, og så fremdeles. Uanset hvor avanceret denne funktion er, danner den til sidst et tal, som siger noget om, hvor god en position er.

Så det næste trick: Siden ingen maskine kan søge gennem hele træet, må vi give den en regel om en maksimal søgedybde. ”Se ikke mere end fem træk frem,” kunne reglen lyde. Men denne regel giver os et alvorligt problem: Det femte træk frem i tid kan indebære, at maskinen bruger sin dronning til at slå en af modstanderens bønder, hvilket er fornuftigt. Men hvis nu denne bonde bliver beskyttet af en anden bonde, sådan så modstanderen ender med at tage maskinens dronning med en bonde i træk seks? Dette vil maskinen ikke kunne tage højde for og vil derfor evaluere positionen fem træk frem i tid helt forkert. For at løse dette problem blev der udviklet en algoritme ved navn *quiescence-søgning*, og den indebærer, at maskinen først stopper sin søgning, når den når en stabil position, hvor alvorlige ting som ”bonde slår dronning” ikke kan ske.

Og så til det sidste trick: Regnekraft er blandt de mest værdifulde ressourcer, søgealgoritmer har. For at spare regnekraft er det vigtigt at kunne indse, at nogle fremtidige positioner er helt håbløse og ikke værd at undersøge. For at se, hvilke fremtidige positioner der er værd at forfølge ned gennem træet, og hvilke grene der er uinteressante, anvendes en algoritme, der hedder *alpha-beta-pruning*. ”Pruning” betyder beskæring, og

hensigten er simpelthen rent virtuelt at skære de uinteressante grene af træet – præcis som man skærer de dårlige grene af et frugttræ.

Disse ingredienser – minimax med fast dybde, quiescence-søgning og alpha-beta-beskæring – udgør hjørnestenene i alle skakcomputere, herunder også *Stockfish*, som bruges i skakudsendelser i fjernsynet. I dag kan Stockfish slå en hvilken som helst menneskelig skakspiller kun ved at anvende den regnekraft, man finder i en almindelig iPhone – men al denne regnekraft har ikke altid været tilgængelig for os.

Professor Edward Fredkin var ophavsmand til mange vigtige fremskridt inden for kunstig intelligens. Alligevel er han måske mest kendt for at have givet navn til en forskningspris, som blev indstiftet af Carnegie Mellon-universitetet i 1980. Fredkin-prisen, der var på 100.000 amerikanske dollar, skulle motivere dataloger til at skabe en computer, som kunne slå verdens bedste skakspiller. Men den forsker, der endte med at modtage prisen, havde aldrig haft ambitioner om at vinde den. Der var snarere tale om et resultat af tilfældigheder og en ph.d.-grad, som tog en anden retning end først planlagt. Helten i historien er den unge ph.d.-studerende Feng-hsiung Hsu, som ikke havde meget tilovers for kunstig intelligens. Hsu begyndte at arbejde på sit projekt ved Carnegie Mellon-universitetet i 1985 og har efterfølgende fortalt, at han ikke som sådan anså kunstig intelligens for at være "bullshit", men havde "set en del forskning i kunstig intelligens, som virkelig fortjente bullshit-mærkatet".⁸ Hsu havde heller ingen særlig interesse for skakspillende computere, men var snarere en praktisk anlagt datalog med en forkærlighed for ingeniørproblemer – særligt dem, der angik de mindste byggeklodser i computere, de såkaldte chips. Enkelt forklaret er en chip en lillebitte plade med et elektrisk kredsløb på, som sammen med andre små dele udgør en computer. Og var der noget, Hsu kunne lide, så var det at lave bedre og bedre chips.

I dag kan du ved at købe den rigtige app eller det rigtige program gøre din mobil eller din PC til skakmester, selvom ingen af de to er bygget

specifikt til at spille skak. Det er selve charmen ved digitale computere, at de kan køre et hvilket som helst program uden at være bygget til det. Men i slutningen af 1980'erne stod skakcomputere over for en stor udfordring, når det gjaldt beregningskraft: Selv med de tre tricks, vi har gennemgået, bliver en skakcomputer bedre, jo flere potentielle fremtidige stillinger den har kapacitet til at udforske. De bedste skakcomputere var derfor designet og bygget specifikt til at kunne søge hurtigt. Og én vigtig ingrediens i skræddersyede computere er gode chips. Omstændighederne gjorde derfor, at Hsu og hans medstuderende blev rekrutteret – omend noget modvilligt – til at deltage i skakturneringer for computere.⁹ Den første turnering, Hsu deltog i, fandt sted med syv ugers varsel og indebar derfor en stor mængde udvikling under lidt for meget pres og med lidt for lidt tid til at teste. Sammen med resten af sit hold deltog han med et program, som de for at understrege, at programmet ikke var færdigtestet, kaldte for *ChipTest*. Resultatet var middelmådigt, men allerede året efter vandt de en overlegen sejr. På dette tidspunkt kunne *ChipTest* søge gennem 500.000 fremtidige træk i sekundet. Vejen derfra og til virkeligt gode skakcomputere skulle komme til at gå via en kraftig forøgelse af det antal skaktræk, maskinen var i stand til at gennemse.

Da Hsu først begyndte at arbejde på det, der senere skulle blive til *ChipTest*, blev det anslået, at det ville være muligt at skabe en kunstig verdensmester i skak, hvis man kunne få hardwaren til at arbejde tusind gange hurtigere. Denne ingeniørmæssige udfordring var nok til at tænde en gnist i Hsu, og hans motivation lå i at finde ud af, om en stor øgning i hastigheden virkelig var nok til at løse skakproblemet. At slå verdens bedste skakspiller anså Hsu som en eventuel bonus.

Efter at have afsluttet sin ph.d.-afhandling, som endte med at handle om *ChipTest* og skakturneringer, blev Hsu ansat hos IBM. Der blev *ChipTest* videreudviklet til *Deep Thought*, som utvivlsomt lyder mere Terminator-agtigt end *ChipTest*.¹⁰ I 1989 udfordrede *Deep Thought* den

regerende verdensmester Garri Kasparov for første gang. Kasparov vandt med lethed kampens to partier.

Hsus udfordring var at finde en smart måde at søge gennem tilstrækkeligt mange mulige fremtidige træk på, og netop dét lykkedes ham. I samarbejde med mange andre ingeniører brugte han et helt ph.d.-forløb efterfulgt af et årti med udvikling på det. Arbejdet kulminerede i et bæst af en skakcomputer ved navn *Deep Blue*, som skulle blive den første maskine i verden, der besejrede en regerende verdensmester. Nu ville det være utroligt tilfredsstillende at kunne fortælle dig om det geniale kneb, Hsu fandt på, og som gjorde, at computere kunne tage skridtet til at blive skakmestre. Men sådan et kneb findes desværre ikke. Det, Hsu præsterede i løbet af de år, han arbejdede med skakcomputere, var solid ingeniørkunst kombineret med godt algoritmedesign. Han byggede specialtilpassede chips og arbejdede intenst på at få skakalgoritmerne til at blive så lidt beregningstunge som overhovedet muligt. Desuden samarbejdede han med stormestrene Miguel Illescas, John Fedorowicz og Nick de Firmian om at finjustere evalueringsfunktionerne. I tillæg til avancerede algoritmer og den regnekraft, Hsu havde brugt hele sin karriere på at give den, indeholdt Deep Blue altså også store mængder moderne skakvisdom. Det hele gav bonus, og den 10. februar 1996 besejrede Deep Blue den regerende verdensmester Garri Kasparov. Alligevel er det ikke den dato, der er blevet berømt: Ikke før foråret 1997 mødtes Kasparov og Deep Blue i en kamp, hvor de officielle regler for skakturneringer blev overholdt. Efter seks partier fordelt over otte dage stod Deep Blue tilbage som sikker vinder. Den 11. maj 1997 blev en mærkedag i AI-historien: For første gang måtte menneskene overlade skaktronen til maskinerne.

Tænkningens bananflue

Garri Kasparov havde spillet mod skakcomputere flere gange tidligere. Allerede i 1985 spillede Kasparov simultant mod 32 af verdens bedste

skakcomputere og besejrede dem alle. Som Kasparov efterfølgende har sagt: "For mig var det min guldalder. Maskinerne var svage, og mit hår var kraftigt!"¹¹ Tolv år senere havde Kasparov nok at gøre med at koncentrere sig om én computer – som altså endte med at besejre ham. På det tidspunkt havde Kasparov været verdensmester i over ti år. Før kampen mod Deep Blue havde han bedt om at få lov til at studere andre partier, Deep Blue havde spillet, men det afslog IBM. Kasparov var vant til at vurdere sine modstanderes strategi, observere deres kropssprog og se dem i øjnene. At sidde over for Deep Blue gav ham en ny følelse af noget urovækkende. Jeg forestiller mig, at fabriksarbejdere, som blev overhalet af dampmaskinen under den første industrielle revolution, måske havde den samme følelse. På forsiden af Newsweek blev opgøret mellem Kasparov og Deep Blue omtalt som "the brain's last stand". Efter 1997 spekulerede Kasparov på, om Deep Blue var uovervindelig, og om skakspillet nu var løst, besejret, forbi. Han siger selv, at han følte frygt og tvivl. Måske følte han til og med en eksistentiel angst. Som Kasparov har påpeget, havde Deep Blue det nok ikke på samme måde.¹² Hsu havde et helt andet syn på sagen. Han sagde, at det "i grunden var to mennesker i forskellige roller, der konkurrerede: mennesket som udøver og mennesket som værktøjsmager".¹³ Deep Blue havde hverken intuition eller erfaring, hvilket Kasparov derimod havde imponerende meget af. Deep Blue vandt på regnekraft, avancerede søgealgoritmer og evalueringsfunktioner udviklet af menneskelige eksperter over mange årtier. Kort sagt beregnede Deep Blue sig til sejren. I dag bliver ingen skræmt eller overrasket over, at computere besejrer skakmestre, lige så lidt som vi bliver bragt ud af fatning af, at en motorcykel kan køre hurtigere, end Usain Bolt kan løbe. Deep Blue gennem søgte svimlende 200 millioner positioner i sekundet. Denne proces ligner ikke rigtigt det, vi mennesker gør, når vi spiller skak – præcis ligesom vores fly ikke har vinger til at baske med. Deep Blues metode er det, man inden for programmering kalder *brute force*: Med rå kraft regnede den sig frem til det bedste svar.

Maskinernes dominans i skakverdenen har dog ikke gjort spillet mindre interessant for os: Selvom enhver smartphone kan køre skakapps, der er stærkere, end Deep Blue var i 1997, følger vi de menneskelige skakmestre med spænding, og vi spiller mere skak end nogensinde før. I 2018 påpegede Kasparov, at skak, ud over at være et fascinerende spil, også kan anses som en form for *tænkningens bananflue*. For præcis på samme måde som *Drosophila melanogaster* – i daglig tale bananfluen – er en nyttig organisme for genforskere, er skakken blevet ”et laboratorium for kognition”. Faktisk tror jeg, at vi roligt kan sige, at skak er det mest udførligt studerede område inden for kunstig intelligens, og i 2020’erne bruger vi stadig skak til at udvikle og undersøge maskinel intelligens.

For mennesker handler skakspil om abstrakt tænkning, om strategi, mønstergenkendelse, dyb koncentration og ofte også om psykologi. Mennesker bliver ikke skakmestre ved at evaluere mange millioner stillinger i sekundet. Der er noget andet på spil, noget, vi mennesker har *forstået*. Og det er jo netop det, vi er ude efter i intelligente maskiner. Dette mystiske, som vi kalder *forståelse*.

Universets bedste skakspiller

Selvom Deep Blue var en fantastisk ingeniørbedrift og spillede skak på en intelligent måde, ville kun de færreste af os tænke på maskinen som *intelligent*. Denne følelse er et fremragende eksempel på et historisk fænomen, som er så velkendt, at det har fået sit eget navn: *The AI effect*. Denne ”AI-effekt” beskriver det fænomen, vi observerer, omtrent hver gang det lykkes nogen at bygge en computer, som løser et problem, vi troede krævede intelligens – for eksempel at spille skak – og vi bagefter ombestemmer os og siger: ”Åh ja, men *det der* er jo ikke intelligens! Det er jo bare en beregning!” Det er ikke godt at sige, hvad der skal til, for at vi mennesker bliver overbevist om en maskines intelligens. Skal den være tilpasningsdygtig, skal den have bevidsthed, skal den beregning, den

udfører, være uforståelig for os, lidt på samme måde som menneskehjernen stadig er et mysterium for os? Tænk lidt over det. Hvis du lander på det sidste kriterie, altså at maskinen skal gøre noget, der er lidt mystisk, er du ikke alene.

I 1980'erne analyserede skakmesteren og filosofen Eliot Hearst, hvor forskelligt mennesker og maskiner opfører sig, når de spiller skak.¹⁴ Skakcomputerne bruger smarte algoritmer og derefter lynhurtige søgninger gennem enorme antal mulige udfald af potentielle træk for at udregne vind sandsynligheden af hvert af dem. Menneskelige skakspillere gør derimod noget helt andet, hvilket også Hearsts eksperimenter viste. Det, vi mennesker gør, er både langt hurtigere og mere energibesparende: Vi genkender mønstre på skakbrættet. På den måde kan de bedste skakspillere blandt os aflæse af en position, hvilken strategi der egner sig bedst. Mennesker, som virkelig har forstået skak, har med andre ord forstået noget overordnet, noget konceptuelt, som gemmer sig i brikkernes kombinationer – og som desuden har meget lidt at gøre med at søge gennem millioner af mulige træk. Hearst mente, at medmindre computere lærte at genkende mønstre og forstå abstrakte koncepter på samme måde, ville de aldrig kunne spille skak som mennesker.

Mens vi kan tænke, at traditionelle skakcomputere som Deep Blue vinder gennem rå styrke, og at deres kapacitet til at evaluere et enormt antal spilpositioner kompenserer for deres manglende intuition for spillet, kan vi også tænke på det hele helt omvendt: Siden vi mennesker ikke har kapacitet til at tænke meget mere end ti mulige træk frem, har vi været tvunget til at udvikle en anden evne til at spille skak, nemlig netop denne intuition. I de senere år har udviklingen inden for kunstig intelligens ført til skakcomputere, som behersker en kombination af de to, altså både hurtige søgninger og en mønstergenkendelse, som minder om intuition for spillet.

Den 5. december 2017 blev verden introduceret for *AlphaZero*. Det er en skakcomputer af den mest moderne slags, skabt af et af de ledende

miljøer inden for forskning i kunstig intelligens, nemlig firmaet DeepMind (som blev opkøbt af Google i 2014). Det havde ikke taget årtier at udvikle AlphaZero; faktisk havde den ved at spille mod sig selv i bare 24 timer *lært sig* skak på et overmenneskeligt niveau. DeepMind kunne fortælle, at AlphaZero ikke bare spillede på et for mennesker uopnåeligt niveau, men også udkonkurrerede førende skakcomputere som Stockfish. AlphaZero lærte sig at spille skak ved at træne for sig selv. Det er grunden til, at der indgår et "zero" i navnet: Den kunne ingenting – zero – da den begyndte at spille mod sig selv. Og i løbet af et døgn havde den tilsyneladende udviklet sig til at blive universets bedste skakspiller – medmindre nogen også har opfundet skak på en anden planet end vores. Men vi må huske på, at et døgn i mennesketid kan være noget helt andet for en computer. DeepMind – og alle store techvirksomheder – har adgang til ufatteligt store beregningsressourcer. Den nuværende version af AlphaZero har perfektioneret sine skakevner ved at have spillet 44 millioner partier. Det ville have taget måneder på den slags computere, de fleste af os har råd til at købe.

Hvor Deep Blue havde brug for regnekraft og menneskeskabte algoritmer, lærte AlphaZero gennem sine 44 millioner partier dybest set at udvikle intuition. AlphaZero udvælger ikke sit næste træk, men beslutter, hvilke træk den mener er værd at se nærmere på. Baseret på denne udvælgelse bestemmes derefter næste træk ved hjælp af samme type søgetræ, som Deep Blue og Stockfish benytter sig af. AlphaZero kombinerer altså en slags *maskinel intuition* for skak med maskinel beregningsevne. Det anslås, at de bedste menneskelige skakspillere er i stand til allerhøjst at foretage nogle hundrede søgninger – altså maksimalt se nogle hundrede mulige fremtidige positioner for sig. Stockfish og andre traditionelle skakcomputere laver i dag millioner af søgninger, mens AlphaZero takket være den skakintuition, den har lært sig, kan "nøjes" med at lave under 100.000 søgninger per stilling.

Det mest interessante ved AlphaZero er måske, at den har udviklet en ikkemenneskelig spillestil. For mens traditionelle skakcomputere er baseret på menneskelig ekspertviden, har AlphaZero lært at spille skak ved at spille frit. Når den træner, spiller den de træk, den selv anser for at være de mest lovende. Derfor er det lykkedes den at opdage strategier, vi mennesker ikke kender til, og menneskelige skakspillere kan lære noget af at spille mod maskinen. Efter min mening er det noget af det bedste, vi kan opnå inden for kunstig intelligens: Maskiner, der lærer os noget nyt og værdifuldt. På den måde er skak blevet noget mere end et laboratorium for intelligens: Spillet har givet maskinerne anledning til at demonstrere, at de ved at prøve sig frem og lave fejl kan tilegne sig *viden*, vi mennesker ikke har og ikke kan give dem.

Maskinernes tid

Har vi i en verden, hvor AlphaZero har udviklet intuition for skak og desuden opdaget strategier i spillet, som vi mennesker ikke havde nogen anelse om, nu den følelse, at der er opstået ægte kunstig intelligens? De fleste, jeg kender, vil svare nej til dette spørgsmål. Hvordan kan det mon være, når vores kunstige skakmestre er så gode, at selveste Magnus Carlsen, verdens bedste skakspiller, tager ved lære af dem? Måske kan skakboksning bruges til at illustrere følelsen. Og ja, skakboksning findes skam i virkeligheden. Som du måske har gættet, indeholder sporten både boksning og skak, og man kan vinde ved at give modstanderen tæv eller ved at slå modstanderen i skakbetydningen af ordet. Og vi ved jo, at en computer, som kører skakprogrammet AlphaZero, hverken har en krop eller mulighed for at rejse sig op, tage boksehandsker på og gå ind i ringen. AlphaZero er "kun" et computerprogram med et meget smalt sæt evner. Den kan spille, men den ved jo ingenting om verden – i hvert fald ikke på samme måde, som vi mennesker forstår, at vi lever i en stor verden fuld af problemer, som ikke har noget med skak at gøre. Indtil nu har ingen

maskiner fået chancen for at stille op i en skakbokseturning, men det er nok snarere af etiske grunde end af praktiske. Maskiner, der kan slå mennesker bevidstløse, findes allerede. Vi kunne sagtens tage en af Boston Dynamics' humanoide robotter, som imponerer os i YouTube-videoer, hvor de slår saltoer og farer gennem forhindringsbaner, installere en version af AlphaZero på deres computerenhed og bruge dem til både skakspil og boksning. Men hvad ville vi så have? Jo, så ville vi have en kombineret bokse-skakspils-robot. Ville den være intelligent og tilpasningsdygtig som et menneske? Nej, der ville stadig mangle noget. Det er ikke godt at sige, hvor mange maskinelle evner vi måtte kombinere, før vi følte, at maskinerne begyndte at nærme sig den menneskelige evne til generalisering.

Alligevel betyder det *ikke*, at maskiner er os mennesker underlegne, for det er ikke længere os, der sidder på den intellektuelle trone. Inden udgangen af 2018 havde computere slået os i de fleste spil: skak, Texas Hold'em-poker og mindre kendte spil som shogi, go og Dota 2. Sidstnævnte er et vældigt populært og ikke mindst kompliceret multiplayer-spil, som spilles online med hold bestående af flere spillere, der må planlægge sammen og forsøge at narre deres modstandere. Den 6. juni 2018 lykkedes det for første gang flere spillere med kunstig intelligens at samarbejde om at slå et hold bestående af menneskelige spillere. Den betryggende nyhed er, at disse kunstigt intelligente spillere også er villige til at arbejde sammen med menneskelige spillere, altså at være partnere i stedet for modstandere. Grunden til, at maskiner vinder over os i vores spil, er ikke, at de har et iboende ønske om at udkonkurrere mennesker: Maskinerne besejrer os, fordi vi beder dem om det. Det er ikke maskinerne, men deres skabere, altså forskere som jeg selv, som bestræber sig på, at maskinerne skal udkonkurrere mennesker på alle de områder, vi falder over. En eventuel frygt, vi måtte have for maskiners *handlinger*, er derfor grundlæggende set én af to ting: Det er enten en frygt for

menneskers ønsker eller en frygt for vores manglende forståelse af, hvad vi beder maskinerne om. Som forsker i kunstig intelligens er jeg ikke bekymret for, at min maskine kan have en skjult motivation for at slå mig i et spil. Jeg er bekymret for noget helt andet.

Vi lever i en guldalder for udviklingen af kunstig intelligens. De sidste år har vi set, hvordan teknikker baseret på kunstig intelligens er blevet i stand til at løse opgaver bedre og bedre, og fremskridtene har føltes nærmest ustoppelige. I 2017 var AlphaZero det mest banebrydende inden for kunstig intelligens, men det har ændret sig på bare få år. Kunstig intelligens har virkelig fundet vej ud fra laboratorierne, og man behøver ikke forske i kunstig intelligens for at mærke fremskridtene: Når du låser din telefon op med dit ansigt, er det, fordi din telefon har lært sig, hvordan du ser ud, selvom du ændrer frisure, søvnmønster eller belysning. Når du ikke kan lade være med at blive ved med at scrolle ned gennem et socialt medie, er det, fordi platformen har lært, hvordan den skal tilpasse sig til dine præferencer for at holde på din opmærksomhed. Når du læser en tekst, kan du ikke vide, om den er skrevet af et menneske eller en maskine. Og når en arbejder i en af Amazons lagerbygninger i USA bliver fyret, kan det sagtens være, at beslutningen er blevet truffet af en maskine. For kun få år siden kæmpede de bedste chatbots med at holde en intelligent samtale gående. I dag bruges kunstigt intelligente sprogsystemer til at forfatte propaganda, skrive artikler og udvikle computerspil, og chatbots kan underholde mennesker i timelange samtaler. I efteråret 2022 smuglede en kunstner et billede skabt af et kunstigt intelligent program ind i en konkurrence, og maskinen endte med at gå af med sejren, uden at nogen af dommerne anede uråd. På få år er kunstigt intelligente systemer gået fra at være klodsede til at være overmenneskelige.

Samtidig er vi mange forskere, som mener, at kunstig intelligens ikke er i nærheden af at opnå hverken bevidsthed eller generel intelligens. At selv de bedste chatbots er mekaniske papegøjer, som recirkulerer, hvad vi end

har fodret dem med, og at den virkelige udfordring ikke er, at vi kommer til at dele kloden med en anden intelligent race, men snarere at vi er omgivet af systemer, som forstærker vores egne fordomme. Men uanset om udviklingen af kunstig generel intelligens ligger langt, langt ude i fremtiden, er kunstigt intelligent teknologi allerede godt på vej ind i samfundet, i vores liv og i vores lommer. Om få år kan det være, at de fleste af de billeder, du ser på nettet, er skabt af maskiner, og at der kun er ét menneske involveret i mange af de samtaler, du fører online. Og at techvirksomheder påvirker politiske strømninger gennem velplacerede argumenter på sociale platforme lige så effektivt som velbetalte lobbyister i Bruxelles. Derfor tror jeg, at grundlæggende viden om kunstig intelligens hurtigt vil blive afgørende for at forstå meget af det, der sker i vores liv, og for at kunne deltage aktivt i den offentlige debat om, hvordan vi ønsker, at denne teknologi skal påvirke vores samfund.

Lad os begynde med at lytte til Voltaire, som sagde: "Définér dine begreber, ellers vil vi aldrig kunne forstå hinanden."¹⁵ Og det allerførste, vi er nødt til at blive enige om, er en forkortelse: Selvom vi på norsk taler om kunstig intelligens, og det naturligste i verden havde været at forkorte det til "KI", er der mange i Norge, som bruger forkortelsen "AI" – inklusive mig selv. Og det er, på trods af at jeg på Arendalsugen i 2019 fik streng besked af en minister om, at "i Norge siger vi KI!". Forklaringen er, at mange af dem, der arbejder inden for feltet, begyndte at programmere kunstig intelligens, før politikere og myndigheder begyndte at tale om emnet. Vi har talt om *artificial intelligence* med forskere fra andre dele af verden, har læst bøger om *artificial intelligence* på engelsk og har altid anvendt forkortelsen AI. Og fordi jeg er en af disse irriterende rebeller, hedder det i denne bog kunstig intelligens – med forkortelsen AI.

Hvad *er* kunstig intelligens så? Vores egen nationale strategi for kunstig intelligens, som vi har haft, siden den daværende digitaliseringsminister præsenterede den i januar 2020, bruger denne definition: "Kunstigt

intelligente systemer udfører handlinger, fysiske eller digitale, baseret på tolkning og behandling af strukturerede eller ustrukturerede data, i den hensigt at opnå et givet mål. Enkelte KI-systemer kan også tilpasse sig gennem at analysere og tage hensyn til, hvordan tidligere handlinger har påvirket omgivelserne.” Ikke for at være krakilsk, men denne definition passer på alt lige fra mennesker til internetsider, hvor man kan søge en vare frem, som for eksempel biltema.no – men det er (for en gangs skyld) ikke regeringens skyld. Det er, fordi det er vanskeligt at definere ”intelligens”. Men frem for at forville mig ned i et filosofisk kaninhul vil jeg i stedet antage, at du allerede har en mening om, hvad du tænker på som *intelligens*, og så støtter vi os til det. Derefter kan vi tilføje, at *kunstig intelligens er et fagområde inden for datalogi med det formål at udvikle maskiner, der er i stand til at opføre sig intelligent*. Der har vi det, drømmen om målet: at lave intelligente maskiner. Men hvordan skal vi gå frem for at gøre maskinerne intelligente? Grundlæggende set findes der to tilgange, som er så forskellige, at de udgør en dyb filosofisk kløft i fagområdet: de *symbolske* og de *subsymbolske* tilgange.